

15-213 , 20xx 年秋季

攻击实验：理解缓冲区溢出漏洞

The Attack Lab: Understanding Buffer Overflow Bugs

非官方中文翻译版

布置时间	9 月 29 日 (星期二)
截止时间	10 月 8 日 (星期四) 23:59 EDT
最迟提交时间	10 月 11 日 (星期日) 23:59 EDT

译者说明

本文件是对 CMU CS:APP3e Attack Lab 2016 年讲义的非官方中文翻译。函数名、寄存器、命令、地址、十六进制字节、代码及参考文献保持原样。仅可在本课程提供的授权目标程序上练习。

自学版安全提示

运行 `ctarget` 或 `rtarget` 时必须加入 `-q` 参数，避免目标程序尝试联系不存在的评分服务器。例如：`./ctarget -q`。

1 引言

本作业要求你针对两个具有不同安全漏洞的程序构造总计五次攻击。完成本实验后，你将获得以下收获：

- 了解当程序没有充分防范缓冲区溢出时，攻击者可以采用的多种漏洞利用方式。
- 更好地理解如何编写更安全的程序，以及编译器和操作系统用于降低程序脆弱性的若干机制。
- 深入理解 x86-64 机器代码中的栈与参数传递机制。
- 深入理解 x86-64 指令的编码方式。
- 进一步熟悉 GDB、OBJDUMP 等调试工具。

注意

本实验会让你直接接触利用操作系统和网络服务器安全弱点的方法。目的在于帮助你理解程序的运行时行为和这些弱点的本质，从而在编写系统代码时避免它们。我们不认可利用任何其他攻击形式未经授权访问系统资源。

建议把 CS:APP3e 教材第 3.10.3 节和第 3.10.4 节作为本实验的参考资料。

2 实验安排

与往常一样，这是一个个人项目。你将针对为自己定制生成的目标程序构造攻击。

2.1 获取文件

在课程环境中，可通过浏览器访问以下地址获取文件：

```
http://$Attacklab::SERVER_NAME:15513/
```

讲师配置项

`$Attacklab::SERVER_NAME` 是运行 attacklab 服务的机器。讲师需要在 `attacklab/Attacklab.pm` 和 `attacklab/src/build/driverhdrs.h` 中定义它。

服务器会构建你的文件，并以 `targetk.tar` 压缩包返回；其中 `k` 是目标程序的唯一编号。构建和下载可能需要几秒，请耐心等待。

将 `targetk.tar` 保存到受保护的 Linux 目录中，然后执行：

```
tar -xvf targetk.tar
```

该命令会解压出 `targetk` 目录。只应下载一套文件；如果意外下载了多个目标，请选择一个并删除其余目标。

警告

如果在 PC 上使用 WinZip 等工具解压，或让浏览器自动解压，可能会重置可执行文件的权限位。建议直接在 Linux 中用 `tar` 解压。

`targetk` 中包含：

- `README.txt`：说明目录内容。
- `ctarget`：容易受到代码注入攻击的可执行程序。
- `rtarget`：容易受到返回导向编程攻击的可执行程序。
- `cookie.txt`：攻击中用作唯一标识符的 8 位十六进制代码。
- `farm.c`：目标 gadget farm 的源代码，用于构造 ROP 攻击。

- hex2raw：生成攻击字符串的工具。

后续说明假设你已将这些文件复制到受保护的本地目录，并在该目录中执行程序。

2.2 重要规则

以下规则在第一次阅读时可能不易理解，但开始实验后可作为集中参考：

- 必须在与生成目标程序的机器相似的环境中完成作业。
- 不得通过攻击绕过目标程序的验证代码。攻击字符串中供 ret 指令使用的地址，只能指向 touch1、touch2、touch3、注入代码，或 gadget farm 中的 gadget。
- 只能使用 rtarget 中 start_farm 与 end_farm 地址范围内的代码构造 gadget。

3 目标程序

CTARGET 和 RTARGET 都通过标准输入读取字符串，调用的 getbuf 函数如下：

```
unsigned getbuf()
{
    char buf[BUFFER_SIZE];
    Gets(buf);
    return 1;
}
```

Gets 与标准库函数 gets 类似：从标准输入读取字符串，遇到换行符或文件结尾停止，并将字符串和结尾的空字节写入指定位置。这里的目标位置是长度为 BUFFER_SIZE 的数组 buf；BUFFER_SIZE 是生成当前目标程序时确定的编译期常量。

Gets() 和 gets() 无法判断目标缓冲区是否足够大，只会持续复制字节，因此可能越过目标存储空间的边界。输入足够短时，getbuf 正常返回 1：

```
unix> ./ctarget -q
Cookie: 0x1a7dd803
Type string: Keep it short!
No exploit. Getbuf returned 0x1
Normal return
```

输入很长时通常会发生错误：

```
unix> ./ctarget -q
Cookie: 0x1a7dd803
Type string: This is not a very interesting string, but it has the property ...
Ouch!: You caused a segmentation fault!
Better luck next time
```

示例中的 cookie 会与你的值不同。RTARGET 的普通行为相同。缓冲区溢出通常会破坏程序状态并导致非法内存访问；你的任务是精心构造输入，让 CTARGET 和 RTARGET 执行更有意义的行为。这些输入称为攻击字符串（exploit strings）。

两个目标程序支持以下命令行参数：

参数	含义
-h	显示可用命令行参数
-q	不向评分服务器发送结果
-i FILE	从指定文件读取输入，而不是标准输入

攻击字符串通常包含不可打印字节，HEX2RAW 用于把十六进制文本转换为原始字节。详见附录 A。

关键注意事项

- 攻击字符串中间不能包含字节 0x0a；它是换行符的 ASCII 编码，Gets 遇到后会认为输入已经结束。
- HEX2RAW 要求每个字节写成两位十六进制，并用一个或多个空白分隔。字节 0 必须写成 00。要构造 0xdeadbeef，应输入 ef be ad de；小端序要求反转字节顺序。

课程服务器版本在成功后会通知评分服务器。原讲义示例为：

```
unix> ./hex2raw < ctarget.l2.txt | ./ctarget
Cookie: 0x1a7dd803
Type string:Touch2!: You called touch2(0x1a7dd803)
Valid solution for level 2 with target ctarget
PASSED: Sent exploit string to server to be validated.
NICE JOB!
```

本仓库自学版

请把示例中的目标命令改为 ./ctarget -q 或 ./rtarget -q。这样会在本地验证结果，但不会尝试连接评分服务器。

阶段	程序	Level	方法	函数	分值
1	CTARGET	1	CI	touch1	10
2	CTARGET	2	CI	touch2	25
3	CTARGET	3	CI	touch3	25
4	RTARGET	2	ROP	touch2	35
5	RTARGET	3	ROP	touch3	5

图 1：Attack Lab 阶段汇总。CI 表示代码注入，ROP 表示返回导向编程。

评分服务器会测试攻击字符串并更新记分板。课程环境中的记分板地址为：

```
http://$Attacklab::SERVER_NAME:15513/scoreboard
```

与 Bomb Lab 不同，本实验输入错误不会扣分。前 3 个阶段是针对 CTARGET 的代码注入攻击，后 2 个阶段是针对 RTARGET 的 ROP 攻击。

4 第一部分：代码注入攻击

前三个阶段攻击 CTARGET。该程序保证不同运行之间的栈位置保持一致，并允许把栈上的数据当作可执行代码。这使得攻击字符串可以包含可执行指令的字节编码。

4.1 Level 1

Phase 1 不注入新代码，而是将控制流重定向到一个已有过程。getbuf 由 CTARGET 中的 test 函数调用：

```
void test()
{
    int val;
    val = getbuf();
    printf("No exploit. Getbuf returned 0x%x\n", val);
}
```

正常情况下，getbuf 执行 ret 后会回到 test 的 printf。目标是让它改为执行 touch1：

```
void touch1()
{
    vlevel = 1;          /* Part of validation protocol */
    printf("Touch1!: You called touch1()\n");
    validate(1);
    exit(0);
}
```

你的攻击字符串应使 getbuf 返回时跳到 touch1，而不是 test。即使覆盖了本阶段无关的其他栈内容也没有关系，因为 touch1 会直接退出程序。

建议

- 使用 objdump -d 反汇编 CTARGET；本阶段所需信息都能从反汇编中获得。
- 把 touch1 起始地址的字节表示放到正确位置，使 getbuf 末尾的 ret 把控制流转移到 touch1。
- 注意小端字节序。
- 可以用 GDB 单步执行 getbuf 最后的几条指令，确认控制流是否正确。
- buf 在 getbuf 栈帧中的位置取决于 BUFFER_SIZE 和 GCC 的栈分配策略，需要查看反汇编确定偏移。

4.2 Level 2

Phase 2 要求在攻击字符串中注入少量代码，并让 CTARGET 执行 touch2：

```
void touch2(unsigned val)
{
    vlevel = 2;          /* Part of validation protocol */
    if (val == cookie) {
        printf("Touch2!: You called touch2(0x%.8x)\n", val);
        validate(2);
    } else {
        printf("Misfire: You called touch2(0x%.8x)\n", val);
        fail(2);
    }
    exit(0);
}
```

除了跳转到 touch2，还必须让 touch2 看起来像是接收了 cookie 参数。

建议

- 把注入代码的地址放在正确位置，使 getbuf 的 ret 把控制流转移到注入代码。
- x86-64 的第一个函数参数通过寄存器 %rdi 传递。

- 注入代码应把 cookie 写入 %rdi，再用 ret 转移到 touch2 的第一条指令。
- 不要在注入代码中使用 jmp 或 call，它们的目标地址编码较难构造。即使不是常规函数返回，也统一使用 ret 转移控制流。
- 参阅附录 B，了解如何利用工具得到指令序列的字节编码。

4.3 Level 3

Phase 3 同样是代码注入攻击，但要传递字符串参数。CTARGET 中的 hexmatch 和 touch3 可表示为：

```
/* Compare string to hex representation of unsigned value */
int hexmatch(unsigned val, char *sval)
{
    char cbuf[110];
    /* Make position of check string unpredictable */
    char *s = cbuf + random() % 100;
    sprintf(s, "%.8x", val);
    return strncmp(sval, s, 9) == 0;
}

void touch3(char *sval)
{
    vlevel = 3;          /* Part of validation protocol */
    if (hexmatch(cookie, sval)) {
        printf("Touch3!: You called touch3(\"%s\")\n", sval);
        validate(3);
    } else {
        printf("Misfire: You called touch3(\"%s\")\n", sval);
        fail(3);
    }
    exit(0);
}
```

目标是让 CTARGET 执行 touch3，并让它认为传入的参数是 cookie 的字符串表示。

建议

- 攻击字符串中要包含 cookie 的 8 位十六进制字符，按最高有效位到最低有效位排列，不带 0x 前缀。
- C 字符串由字符字节序列和结尾的 0 字节构成。可在 Linux 上执行 man ascii 查询字符编码。
- 注入代码应把 %rdi 设置为该字符串的地址。
- hexmatch 和 strncmp 会在栈上压入数据，覆盖 getbuf 缓冲区所在内存的一部分，因此必须谨慎选择 cookie 字符串的放置位置。

5 第二部分：返回导向编程

对 RTARGET 进行代码注入比 CTARGET 困难得多，因为它采用两种机制阻止注入：

- 使用地址随机化，使栈位置每次运行都不同，因此无法预测注入代码的位置。
- 将栈所在内存标记为不可执行，即使能跳到注入代码开头，也会发生段错误。

返回导向编程 (Return-Oriented Programming, ROP) 通过执行程序中已有代码而不是注入新代码来完成有用操作 [1, 2]。它寻找以 `ret` 结尾的一小段指令序列，每段称为 `gadget`。栈中依次存放 `gadget` 地址，前一个 `gadget` 末尾的 `ret` 会跳到下一个 `gadget`。

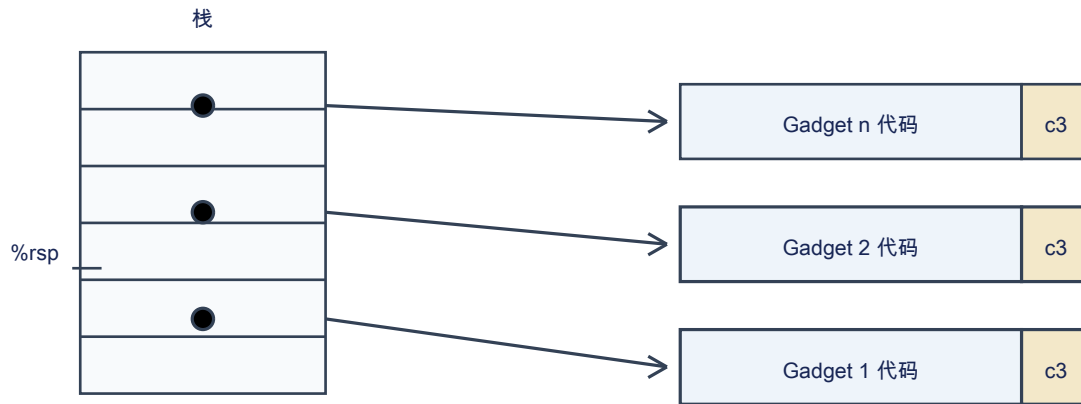


图 2：在栈上设置 gadget 执行序列。字节 0xc3 编码 `ret` 指令。

编译器生成的函数尾部可能包含有用 `gadget`，但数量通常不足。x86-64 是面向字节的变长指令集，因此还可以从其他指令字节序列的中间提取 `gadget`。

例如，某个 `rtarget` 含有以下函数：

```
void setval_210(unsigned *p)
{
    *p = 3347663060U;
}
```

其反汇编为：

```
0000000000400f15 <setval_210>:
400f15: c7 07 d4 48 89 c7    movl $0xc78948d4, (%rdi)
400f1b: c3                  retq
```

字节序列 48 89 c7 编码 `movq %rax,%rdi`，紧随其后的 c3 编码 `ret`。函数从 0x400f15 开始，该序列从第 4 个字节开始，因此地址 0x400f18 处存在一个 `gadget`：把 `%rax` 的 64 位值复制到 `%rdi`，然后返回。

RTARGET 的 `gadget farm` 包含许多类似函数。必须只在 `start_farm` 与 `end_farm` 之间寻找 `gadget`。

5.1 Level 2

Phase 4 要在 RTARGET 上使用 `gadget` 重复 Phase 2 的目标。允许使用以下指令，并且只使用前 8 个 x86-64 寄存器 (`%rax ~ %rdi`)：

- `movq`：编码见图 3A。
- `popq`：编码见图 3B。

- `ret`：单字节编码 `0xc3`。
- `nop`：单字节编码 `0x90`，只让程序计数器加 1。

建议

- 所需 gadget 都位于 `start_farm` 与 `mid_farm` 之间。
- 本阶段只需两个 gadget。
- `popq` 会从栈中弹出数据，因此攻击字符串需要同时包含 gadget 地址和数据。

5.2 Level 3

Phase 5 要对 `RTARGET` 执行 ROP 攻击，以 `cookie` 字符串指针为参数调用 `touch3`。它明显比 Phase 4 更困难。官方解法需要 8 个 gadget，其中有的会重复使用；本阶段在原课程中只占 5 分，可视为挑战题。

Phase 5 可以使用 `start_farm` 与 `end_farm` 之间的全部 gadget。除了 Phase 4 的 gadget，还会使用图 3C 中的 `movl` 编码，以及图 3D 中不改变寄存器或内存值的两字节功能性 `nop`，例如 `andb %al,%al`。

建议

- 复习 `movl` 指令对寄存器高 4 字节的影响，参见教材第 183 页。
- 官方解法需要 8 个 gadget，但并非每个 gadget 都唯一。

图 3 常用指令的字节编码

A. movq 指令编码

源/目标	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
%rax	48 89 c0	48 89 c1	48 89 c2	48 89 c3	48 89 c4	48 89 c5	48 89 c6	48 89 c7
%rcx	48 89 c8	48 89 c9	48 89 ca	48 89 cb	48 89 cc	48 89 cd	48 89 ce	48 89 cf
%rdx	48 89 d0	48 89 d1	48 89 d2	48 89 d3	48 89 d4	48 89 d5	48 89 d6	48 89 d7
%rbx	48 89 d8	48 89 d9	48 89 da	48 89 db	48 89 dc	48 89 dd	48 89 de	48 89 df
%rsp	48 89 e0	48 89 e1	48 89 e2	48 89 e3	48 89 e4	48 89 e5	48 89 e6	48 89 e7
%rbp	48 89 e8	48 89 e9	48 89 ea	48 89 eb	48 89 ec	48 89 ed	48 89 ee	48 89 ef
%rsi	48 89 f0	48 89 f1	48 89 f2	48 89 f3	48 89 f4	48 89 f5	48 89 f6	48 89 f7
%rdi	48 89 f8	48 89 f9	48 89 fa	48 89 fb	48 89 fc	48 89 fd	48 89 fe	48 89 ff

B. popq 指令编码

操作	%rax	%rcx	%rdx	%rbx	%rsp	%rbp	%rsi	%rdi
popq R	58	59	5a	5b	5c	5d	5e	5f

C. movl 指令编码

源/目标	%eax	%ecx	%edx	%ebx	%esp	%ebp	%esi	%edi
%eax	89 c0	89 c1	89 c2	89 c3	89 c4	89 c5	89 c6	89 c7
%ecx	89 c8	89 c9	89 ca	89 cb	89 cc	89 cd	89 ce	89 cf
%edx	89 d0	89 d1	89 d2	89 d3	89 d4	89 d5	89 d6	89 d7
%ebx	89 d8	89 d9	89 da	89 db	89 dc	89 dd	89 de	89 df
%esp	89 e0	89 e1	89 e2	89 e3	89 e4	89 e5	89 e6	89 e7
%ebp	89 e8	89 e9	89 ea	89 eb	89 ec	89 ed	89 ee	89 ef
%esi	89 f0	89 f1	89 f2	89 f3	89 f4	89 f5	89 f6	89 f7
%edi	89 f8	89 f9	89 fa	89 fb	89 fc	89 fd	89 fe	89 ff

D. 两字节功能性 nop 指令编码

操作	%al	%cl	%dl	%bl
andb R,R	20 c0	20 c9	20 d2	20 db
orb R,R	08 c0	08 c9	08 d2	08 db
cmpb R,R	38 c0	38 c9	38 d2	38 db
testb R,R	84 c0	84 c9	84 d2	84 db

图 3：指令字节编码，所有值均为十六进制。

附录 A 使用 HEX2RAW

HEX2RAW 接收十六进制格式的字符串，每个字节用两位十六进制数表示。例如，字符串 012345 可以写成：

```
30 31 32 33 34 35 00
```

十进制数字 x 的 ASCII 编码为 0x3x，字符串以空字节结束。传给 HEX2RAW 的字节必须用空格或换行分隔。建议在调试时用换行划分攻击字符串的不同部分。

HEX2RAW 支持 C 风格块注释，可以标注攻击字符串的不同区域：

```
48 c7 c1 f0 11 40 00 /* mov $0x40011f0,%rcx */
```

注释起止标记 /* 与 */ 两侧都必须留空格，否则可能无法正确忽略。假设十六进制攻击字符串保存在 exploit.txt，可采用以下方式：

1. 使用管道直接传给目标

```
cat exploit.txt | ./hex2raw | ./ctarget -q
```

2. 先保存原始字节文件，再重定向输入

```
./hex2raw < exploit.txt > exploit-raw.txt
./ctarget -q < exploit-raw.txt
```

也可以在 GDB 中使用：

```
gdb --args ./ctarget -q
(gdb) run < exploit-raw.txt
```

3. 使用目标程序的 -i 参数

```
./hex2raw < exploit.txt > exploit-raw.txt
./ctarget -q -i exploit-raw.txt
```

附录 B 生成机器码字节

利用 GCC 作为编译器、OBJDUMP 作为反汇编器，可以方便地得到指令序列的字节编码。假设 example.s 内容如下：

```
# Example of hand-generated assembly code
pushq    $0xabcdef          # Push value onto stack
addq     $17,%rax           # Add 17 to %rax
movl     %eax,%edx          # Copy lower 32 bits to %edx
```

代码可以混合指令和数据，# 右侧内容是注释。汇编并反汇编：

```
gcc -c example.s
objdump -d example.o > example.d
```

example.d 中会得到：

```
example.o: file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <.text>:
 0: 68 ef cd ab 00      pushq $0xabcdef
 5: 48 83 c0 11         add    $0x11,%rax
 9: 89 c2               mov    %eax,%edx
```

冒号后的十六进制数字就是指令字节。因此，push \$0xABCDEF 的编码为 68 ef cd ab 00，完整序列为：

```
68 ef cd ab 00 48 83 c0 11 89 c2
```

可以将该序列交给 HEX2RAW，也可以加入 C 风格注释提高可读性：

```
68 ef cd ab 00  /* pushq $0xabcdef */  
48 83 c0 11     /* add  $0x11,%rax */  
89 c2          /* mov  %eax,%edx */
```

加入这些注释后，文件仍可作为 HEX2RAW 的有效输入。

参考文献

[1] R. Roemer, E. Buchanan, H. Shacham, and S. Savage. Return-oriented programming: Systems, languages, and applications. ACM Transactions on Information System Security, 15(1):2:1-2:34, March 2012.

[2] E. J. Schwartz, T. Avgerinos, and D. Brumley. Q: Exploit hardening made easy. In USENIX Security Symposium, 2011.